

Deep Reinforcement Learning

Exam preparation

Prof. Dr. Sebastian Peitz

Chair of Safe Autonomous Systems, TU Dortmund

Summer term 2026

Timing and grading

Duration: 25 minutes

- 20 minutes for Lecture and Exercises.
 - Divided roughly 50-50.
 - Slightly larger weight on the lecture than on the exercises.
- 5 minutes related to your programming exercises.

Timing will be strictly enforced

- We start with questions right away.
- The number of questions may vary slightly,
 - Goal: 8-12 questions during the first 20 minutes,
 - plus **at least** 2 questions regarding the programming tasks.
- We stop after exactly 25 minutes.
- 5 minutes of internal discussion and grading.

What can I bring to the exam?

Grading criteria

- The quality of the answers.
- How much support was required from our side.
- How quickly you were able to answer.
 - It's not a contest for speed,
 - but if you need to think about a question for several minutes, this reduces the grade.

Questions regarding the programming tasks:

- Goal: find out whether you understand what you did.
- We will ask you for reasons why you solved certain parts in the way you did.
- If it is clear that you know the code and know what is going on, everything is great!
- If you created the code using an LLM without further thinking about it, you will likely have trouble

No additional material allowed!

answering these questions.

Sample questions related to the lecture

1. Name the central quantities of interest in RL. ([General](#))
2. What are the core ingredients of an MDP? ([MDPs](#))
3. What is the policy evaluation step/algorithm in DP? ([Dynamic Programming](#))
4. How does the policy evaluation procedure work for Monte Carlo sampling (in the tabular case)? ([Monte Carlo methods](#))
5. Explain the SARSA algorithm. ([TD learning / Q-learning](#))
6. When using bootstrapping / TD learning in value function approximation, where does the notion of semi-gradients come from? ([Value function approximation & deep Q-learning](#))
7. What's the additional term that shows up in the expectation when we compute the policy gradient? ([Policy gradients](#))
8. How can you interpret the policy gradient with the advantage function as baseline? What do the two terms $\nabla \log \pi(a|s)$ and $A(s, a)$ represent? ([Policy gradients](#))
9. What are the challenges in TRPO and how does PPO solve them? ([Adv. algorithms](#))
10. Which methods to enhance exploration do you know? Can you explain the core mechanics? ([Exploration](#))
11. How does model predictive control (MPC) work? What is its relation to RL? ([Model-based control](#))

12. Explain the core ideas of Monte Carlo Tree Search (MCTS). (*Model-based control*)
13. What is the general working principle behind conservative Q -learning? (*Offline RL*)

Sample question related to the exercises

Exercise 3.1 (Value iteration/Policy iteration) In reinforcement learning, value iteration and policy iteration are two closely related dynamic programming methods for solving a Markov decision process. Policy iteration alternates between evaluating and improving a policy, while value iteration directly updates the value function toward optimality. We recall the Bellman (expectation) operator for a fixed policy π

$$(T^\pi V)(s) = \sum_a \pi(a|s) \sum_{s'} p(s'|s, a) [r(s, a, s') + \gamma V(s')],$$

and the Bellman optimality operator

$$(T^* V)(s) = \max_{a \in \mathcal{A}} \sum_{s'} p(s'|s, a) [r(s, a, s') + \gamma V(s')].$$

Using the operators T^π and T^* we can describe value iteration and policy iteration as follows:

- a) In value iteration the value function is updated by

$$V(s) \leftarrow (T^* V)(s), \quad \forall s \in \mathcal{S}.$$

- b) In policy iteration the updates of the value function and the policy alternate:

- i) Policy evaluation: For π , compute the value function V^π by solving the Bellman expectation equation

$$V^\pi(s) = (T^\pi V^\pi)(s), \quad \forall s \in \mathcal{S}.$$

- ii) Policy improvement:

$$\pi(s) \leftarrow \arg \max_{a \in \mathcal{A}} \sum_{s'} p(s'|s, a) [r(s, a, s') + \gamma V^\pi(s')].$$

3.1.1 Discuss the trade-offs between value iteration and policy iteration. Your answer should compare:

- a) The computational complexity of a single iteration.
b) The stability of the intermediate policies produced by each algorithm.

Sample questions related to the programming tasks

The following are rather generic, but you can expect questions related to your submissions along these lines:

1. How did you select the hyperparameters of your implementation?
2. In terms of computations, what is the bottleneck in the code?

Can I bring my code / do I have to know my code?

- You are not allowed to bring anything to the exam!
- You do not have to know your code. We will have a laptop to present it to you during the exam, if required.